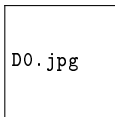


Lecture 35: ICA-4 Revision

Model Solutions + Explanations (Joins, Views, Indexes, Subqueries, CTEs)

Z2004: Database Management Systems



IIT Madras Zanzibar

Tuesday, 16 June 2026

How we will revise ICA-4

- ▶ We'll go question-by-question.
- ▶ For each one: **(1) what the question is really asking, (2) the model SQL, (3) common mistakes.**
- ▶ Goal: you can reproduce the logic under exam time pressure.

Schema used in ICA-4: Student(SID, SName, DeptID), Department(DeptID, DeptName), Course(CID, CName, DeptID), Enroll(SID, CID, Semester, Grade).

Q1 (MCQ): LEFT JOIN meaning

Question: Which JOIN returns all rows from the left table and matching rows from the right table?

Correct answer: (b) LEFT OUTER JOIN

Why (b) is correct

- ▶ Preserves **all** rows from the left table.
- ▶ Unmatched right-side columns become NULL.

Why the other options are wrong

- ▶ (a) **INNER JOIN**: drops left rows with no match.
- ▶ (c) **RIGHT OUTER JOIN**: preserves the **right** table, not the left.
- ▶ (d) **CROSS JOIN**: produces all pairs (Cartesian product).

Q2 (MCQ): What is a VIEW?

Question: Which statement about a VIEW is TRUE?

Correct answer: (b) A view can simplify complex queries

Why (b) is correct

- ▶ A view is a **named, saved query** you can reuse.
- ▶ It improves readability by hiding long joins / filters.

Why the other options are wrong

- ▶ (a) “always stores data physically” is false (most views are not materialized).
- ▶ (c) Views **can** be queried with WHERE.
- ▶ (d) Views do not automatically create indexes.

Q3 (MCQ): When does an index help most?

Question: An index is most helpful when a query frequently:

Correct answer: (a) filters on a column in WHERE

Why (a) is correct

- ▶ Index speeds up searching for matching rows instead of scanning the whole table.

Why the other options are wrong

- ▶ (b) Frequent updates: indexes add maintenance cost → can slow writes.
- ▶ (c) Small tables: full scan is already cheap; index benefit is minimal.
- ▶ (d) DROP TABLE: not a query-optimization use case.

Q4 (MCQ): Correlated subquery

Question: A correlated subquery:

Correct answer: (b) references columns from the outer query

Why (b) is correct

- ▶ The subquery depends on the current outer row (e.g., $e.CID = c.CID$).

Why the other options are wrong

- ▶ (a) “runs once and independent” describes a **non-correlated** subquery.
- ▶ (c) Correlated subqueries **can** appear in WHERE.
- ▶ (d) Not “the same as a join” (can be rewritten as joins sometimes, but conceptually different).

Q5(a): INNER JOIN Student–Department

Task: List SID, SName, DeptName for all students using an **INNER JOIN**.

Model SQL:

```
SELECT s.SID, s.SName, d.DeptName
FROM Student s
JOIN Department d
    ON d.DeptID = s.DeptID;
```

Why this works:

- ▶ JOIN ... ON matches each student to their department.
- ▶ INNER JOIN drops students whose DeptID has no matching Department row.

Common mistakes:

- ▶ Joining on the wrong columns (e.g., SID = DeptID).
- ▶ Selecting DeptID instead of DeptName.

Q6(b): LEFT JOIN courses + enrollment count (Even-2026)

Task: List all courses with number of students enrolled in 'Even-2026'. Show 0 if none.

Model SQL:

```
SELECT c.CID, c.CName,
       COALESCE(COUNT(e.SID), 0) AS EnrollCount
FROM Course c
LEFT JOIN Enroll e
  ON e.CID = c.CID
   AND e.Semester = 'Even-2026'
GROUP BY c.CID, c.CName;
```

Key idea:

- ▶ Keep the semester filter in the ON clause so the LEFT JOIN stays a left join.
- ▶ COUNT(e.SID) becomes 0 for courses with no matching enrollments.

Common mistake:

- ▶ Putting WHERE e.Semester='Even-2026' turns the query into an INNER JOIN (drops empty courses).

Q7(c): CREATE VIEW V_StudentCourse

Task: Create view V_StudentCourse with SID, SName, CID, CName, Semester, Grade.

Model SQL:

```
CREATE VIEW V_StudentCourse AS
SELECT s.SID, s.SName,
       c.CID, c.CName,
       e.Semester, e.Grade
FROM   Enroll e
JOIN   Student s ON s.SID = e.SID
JOIN   Course  c ON c.CID = e.CID;
```

Explanation:

- ▶ A view packages a frequently-used join into a named object.
- ▶ Downstream queries can do: `SELECT * FROM V_StudentCourse WHERE Semester='Even-2026';`

Q8(d): Suggest two helpful indexes

Task: Suggest 2 indexes to speed up parts (a)–(c).

Good choices (examples):

```
CREATE INDEX idx_student_deptid ON Student (DeptID);  
CREATE INDEX idx_enroll_sem_cid ON Enroll (Semester, CID);
```

Why these help:

- ▶ Student (DeptID) speeds up join Student → Department.
- ▶ Enroll (Semester, CID) speeds up filtering Semester='Even-2026' and then matching by CID for course counts.

Alternatives that are also valid:

- ▶ Enroll (CID, Semester) (similar benefit), Enroll (SID) (helps student-based lookups).

Q9(e): Subquery for students in > 3 courses (Even-2026)

Task: Find students enrolled in more than 3 courses in 'Even-2026'. Output SID, SName.

Model SQL (subquery with IN):

```
SELECT s.SID, s.SName
FROM Student s
WHERE s.SID IN (
    SELECT e.SID
    FROM Enroll e
    WHERE e.Semester = 'Even-2026'
    GROUP BY e.SID
    HAVING COUNT(DISTINCT e.CID) > 3
);
```

Explanation:

- ▶ Inner query computes the qualifying SIDs (group + having).
- ▶ Outer query maps those SIDs back to names.

Common mistakes:

- ▶ Forgetting GROUP BY e.SID.
- ▶ Using WHERE COUNT(...) > 3 instead of HAVING.

Q10(Ca): Correlated subquery (model SQL)

Task: List courses where enrollment count in 'Even-2026' is greater than the average across all courses.

Model SQL (correlated):

```
SELECT c.CID, c.CName
FROM Course c
WHERE (
    SELECT COUNT(*)
    FROM Enroll e
    WHERE e.CID = c.CID
        AND e.Semester = 'Even-2026'
) > (
    SELECT AVG(cnt)
    FROM (
        SELECT c2.CID, COUNT(e2.SID) AS cnt
        FROM Course c2
        LEFT JOIN Enroll e2
            ON e2.CID = c2.CID
            AND e2.Semester = 'Even-2026'
        GROUP BY c2.CID
    ) x
```

Q10(Ca): Why it works + how to avoid mistakes

Mental model

- ▶ Compute **this course's** count (correlated subquery using `e.CID = c.CID`).
- ▶ Compute **all courses'** counts, then take the average (derived table `x`).
- ▶ Compare: `this_count > average_count`.

Mistakes that lose marks in exams

- ▶ Writing `AVG(COUNT(*))` in one `SELECT` (nested aggregates need a subquery).
- ▶ Dropping 0-enrollment courses from the average (must use **LEFT JOIN** inside the derived table).
- ▶ Putting the semester filter in `WHERE` after a left join (turns it into an inner join).

Q11(Cb): Recursive CTE (model SQL)

Task: For Prereq(CourseID, PrereqCourseID), list all prerequisites (direct + indirect) for :courseId.

Model SQL (recursive CTE):

```
WITH RECURSIVE AllPrereq(PrereqCourseID) AS (  
    -- Anchor: direct prerequisites  
    SELECT p.PrereqCourseID  
    FROM Prereq p  
    WHERE p.CourseID = :courseId  
  
    UNION  
  
    -- Recursive: prerequisites of prerequisites  
    SELECT p2.PrereqCourseID  
    FROM Prereq p2  
    JOIN AllPrereq ap  
        ON p2.CourseID = ap.PrereqCourseID  
)  
SELECT PrereqCourseID  
FROM AllPrereq;
```

Q11(Cb): How recursion works + exam pitfalls

How it works (in 3 steps)

- ▶ **Anchor:** get direct prerequisites of :courseId.
- ▶ **Recurse:** treat each found prerequisite as a new CourseID and fetch its prerequisites.
- ▶ Stop when no new prerequisite is found.

Mistakes to avoid

- ▶ Joining in the wrong direction (should match p2.CourseID to ap.PrereqCourseID).
- ▶ Using UNION ALL without deduplication (duplicates, and cycles can loop forever).
- ▶ Forgetting the :courseId filter in the anchor (returns prerequisites for everyone).

DBMS note: Some systems omit RECURSIVE; accept the dialect used in your environment.

Wrap-up: ICA-4 patterns to master

- ▶ **Join patterns:** inner vs left; where to place filters to avoid changing join type.
- ▶ **Aggregation patterns:** GROUP BY + HAVING for “more than N”.
- ▶ **Correlated subqueries:** look for outer references.
- ▶ **Recursive CTEs:** anchor + recursive member; stop when no new rows.
- ▶ **Indexes:** choose columns that appear in joins and selective filters.